

Plataforma de Pagamentos na AWS

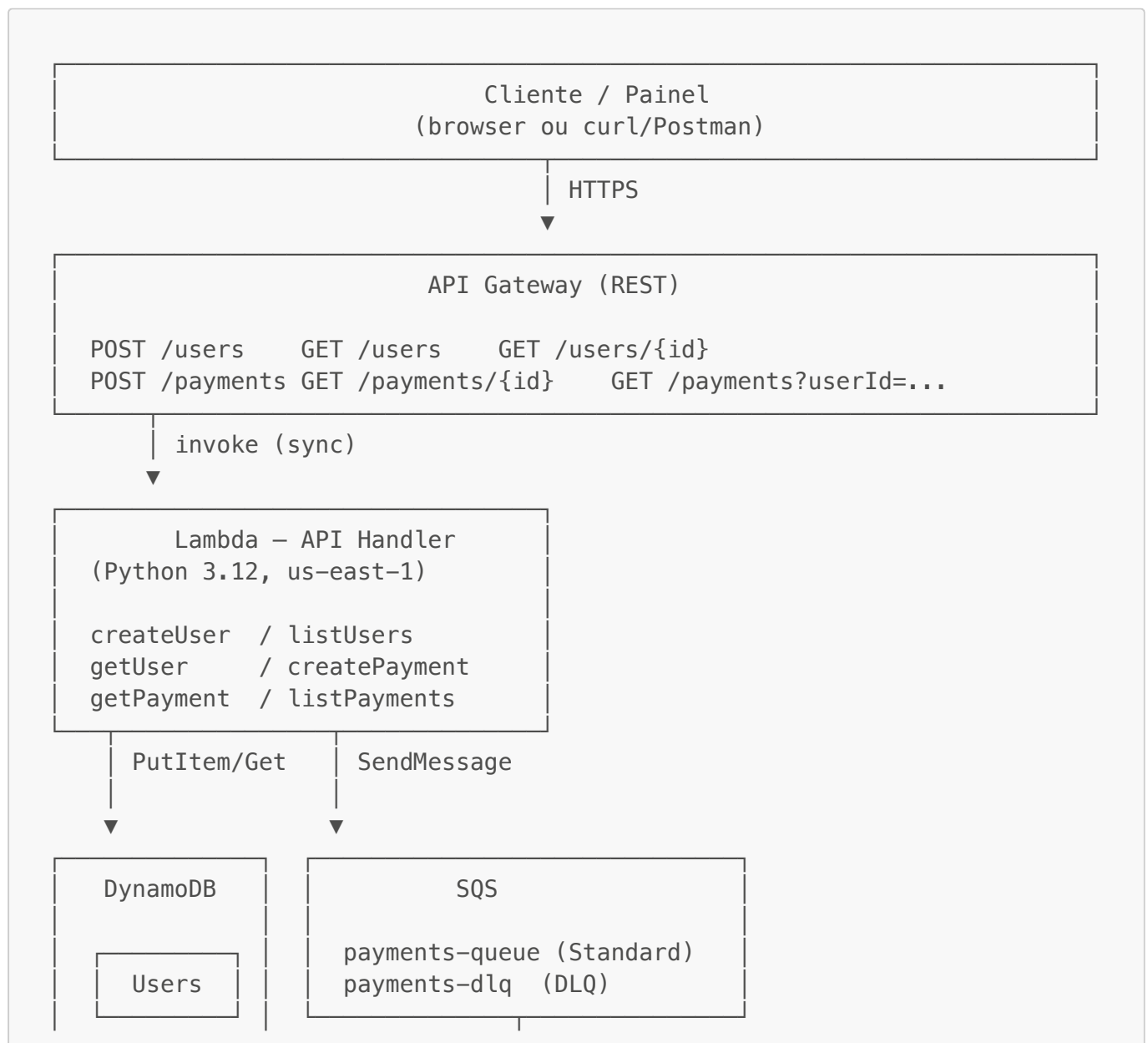
Sumário

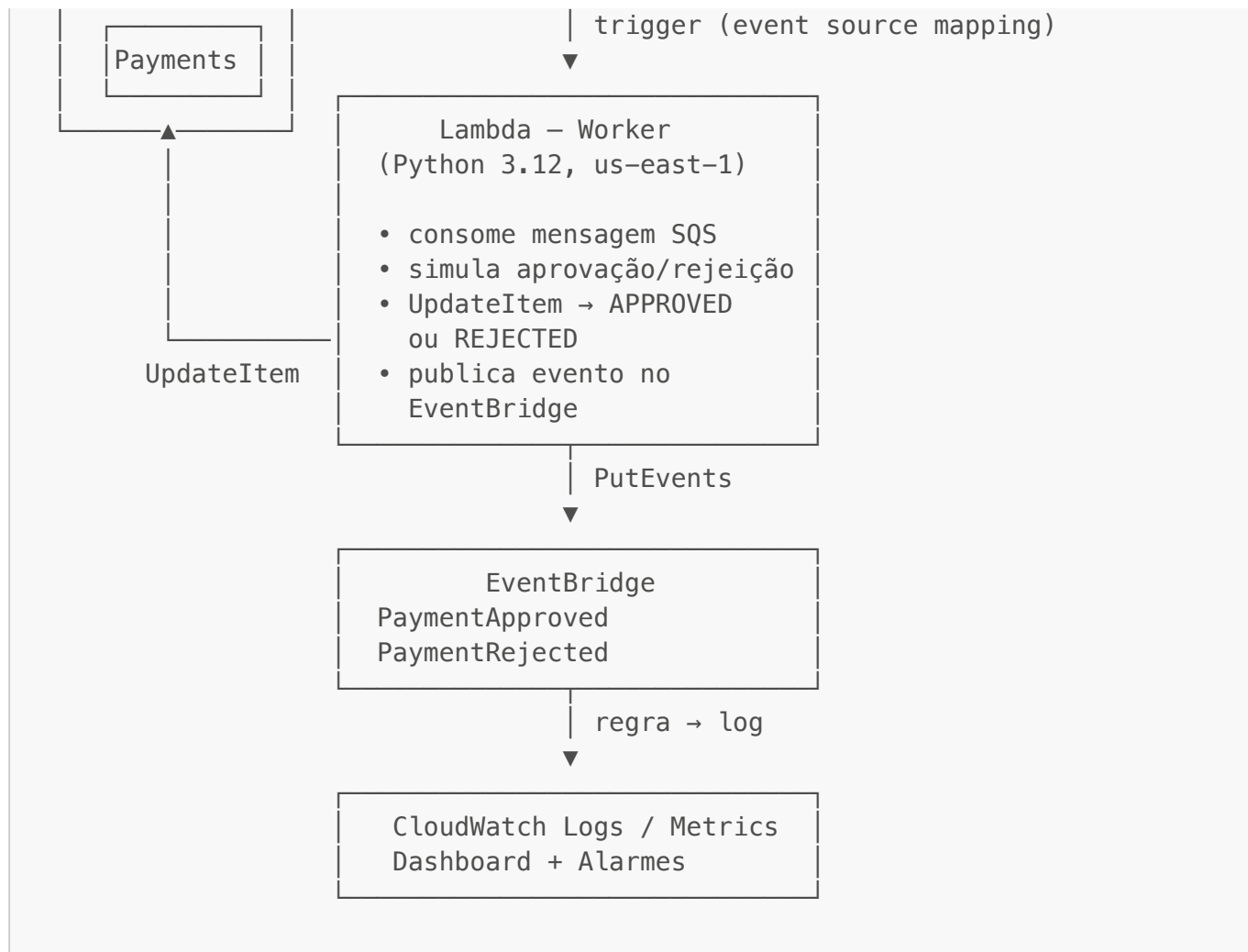
1. [Arquitetura](#)
2. [Infraestrutura criada](#)
3. [Contrato de API](#)
4. [Frontend](#)
5. [Decisões arquiteturais](#)
6. [Testes de carga](#)

Arquitetura

Região AWS: **us-east-1**

Diagrama





Fluxo de dados

1. **Cliente** faz requisição HTTP para o **API Gateway**.
2. API Gateway aciona a **Lambda API Handler** de forma síncrona.
3. Para operações de usuário (**/users**): Lambda lê/grava direto no **DynamoDB > Users**.
4. Para **POST /payments**: Lambda valida o usuário, grava o pagamento com **status=PENDING** no **DynamoDB > Payments** e envia mensagem para **SQS payments-queue**. Responde 202 imediatamente.
5. A **Lambda Worker** é disparada automaticamente pelo trigger SQS, simula o processamento e atualiza o status para **APPROVED** ou **REJECTED** no DynamoDB.
6. O Worker publica evento de domínio no **EventBridge** (**PaymentApproved** / **PaymentRejected**).
7. Regra do EventBridge registra os eventos no **CloudWatch Logs**.
8. Métricas de latência, erros e profundidade de fila ficam visíveis no dashboard do CloudWatch.

Infraestrutura criada

Recurso	Nome	Detalhes
DynamoDB	Users	Partition key: userId (String), on-demand
DynamoDB	Payments	Partition key: paymentId (String), GSI userId-index , on-demand

Recurso	Nome	Detalhes
Lambda	<code>payments-api</code>	Python 3.12, invocada pelo API Gateway
Lambda	<code>payments-worker</code>	Python 3.12, trigger: <code>payments-queue</code> , batch size: 1
API Gateway	<code>payments-api</code>	REST, estágio <code>dev</code> , CORS habilitado
SQS	<code>payments-queue</code>	Standard, DLQ: <code>payments-dlq</code> , max receives: 5
SQS	<code>payments-dlq</code>	Standard — recebe mensagens com 5 falhas consecutivas
EventBridge	<code>payments-events</code>	Event bus customizado para eventos de domínio
EventBridge Rule	<code>log-payment-events</code>	Filtra <code>source: payments.worker</code> → CloudWatch Logs
CloudWatch Log Group	<code>/aws/events/payments</code>	Recebe <code>PaymentApproved</code> e <code>PaymentRejected</code>
CloudWatch Dashboard	<code>payments-dashboard</code>	Latência API Gateway p95, invocações/erros Lambda, profundidade SQS
CloudWatch Alarm	<code>sqs-message-age-too-old</code>	Dispara quando <code>ApproximateAgeOfOldestMessage > 60s</code>
S3	<code>pagamentos-frontend</code>	Static website hosting — painel React
IAM Role	<code>payments-api-role</code>	Least-privilege: DynamoDB + SQS SendMessage
IAM Role	<code>payments-worker-role</code>	Least-privilege: SQS + DynamoDB UpdateItem + EventBridge

Variáveis de ambiente

`payments-api`:

Variável	Valor
<code>USERS_TABLE</code>	<code>Users</code>
<code>PAYMENTS_TABLE</code>	<code>Payments</code>
<code>PAYMENTS_QUEUE_URL</code>	URL da <code>payments-queue</code>

`payments-worker`:

Variável	Valor
<code>PAYMENTS_TABLE</code>	<code>Payments</code>
<code>EVENT_BUS_NAME</code>	<code>payments-events</code>

Contrato de API

Base URL: <https://zll9wu9brj.execute-api.us-east-1.amazonaws.com/dev>

Usuários

POST /users

Cadastra um novo usuário.

Request:

```
{
  "name": "Guilherme",
  "email": "gui@teste.com"
}
```

Response 201:

```
{
  "userId": "usr_8e5dd28b",
  "name": "Guilherme",
  "email": "gui@teste.com",
  "createdAt": "2026-05-16T17:45:37.667748+00:00"
}
```

Erros:

- 400 — campo obrigatório ausente (`name` ou `email`)
 - 409 — email já cadastrado
-

GET /users

Lista todos os usuários cadastrados, ordenados por data de criação.

Response 200:

```
{
  "items": [
    {
      "userId": "usr_8e5dd28b",
      "name": "Guilherme",
      "email": "gui@teste.com",
      "createdAt": "2026-05-16T17:45:37.667748+00:00"
    }
  ],
}
```

```
{  "count": 1}
```

GET /users/{id}

Retorna um usuário pelo ID.

Response 200:

```
{  "userId": "usr_8e5dd28b",  "name": "Guilherme",  "email": "gui@teste.com",  "createdAt": "2026-05-16T17:45:37.667748+00:00"}
```

Erros:

- 400 — id ausente
- 404 — usuário não encontrado

Pagamentos

POST /payments

Cria um novo pagamento e o envia para processamento assíncrono.

Headers opcionais:

- Idempotency-Key: <string> — garante que retries não criem pagamentos duplicados

Request:

```
{  "userId": "usr_8e5dd28b",  "amount": 100,  "currency": "BRL",  "description": "descrição opcional"}
```

Response 202:

```
{  "paymentId": "pay_22f05652",  "userId": "usr_8e5dd28b",}
```

```
"amount": "100",
"currency": "BRL",
"description": "descrição opcional",
"status": "PENDING",
"createdAt": "2026-05-16T17:45:52.507377+00:00"
}
```

Erros:

- 400 — campo obrigatório ausente (`userId`, `amount` ou `currency`)
 - 404 — usuário não encontrado
 - 409 — pagamento duplicado (idempotency-key já usada)
-

GET /payments/{id}

Retorna um pagamento pelo ID, incluindo o status atual.

Response 200:

```
{
  "paymentId": "pay_22f05652",
  "userId": "usr_8e5dd28b",
  "amount": "100",
  "currency": "BRL",
  "description": "teste",
  "status": "APPROVED",
  "createdAt": "2026-05-16T17:45:52.507377+00:00",
  "updatedAt": "2026-05-16T17:45:53.435394+00:00"
}
```

Status possíveis: `PENDING` | `APPROVED` | `REJECTED`

Erros:

- 400 — id ausente
 - 404 — pagamento não encontrado
-

GET /payments?userId={userId}

Lista todos os pagamentos de um usuário.

Query params:

- `userId` (obrigatório)

Response 200:

```
{
  "items": [
    {
      "paymentId": "pay_22f05652",
      "userId": "usr_8e5dd28b",
      "amount": "100",
      "currency": "BRL",
      "description": "teste",
      "status": "APPROVED",
      "createdAt": "2026-05-16T17:45:52.507377+00:00",
      "updatedAt": "2026-05-16T17:45:53.435394+00:00"
    }
  ],
  "count": 1
}
```

Erros:

- 400 — userId ausente

Frontend

O painel administrativo está hospedado publicamente no S3:

<http://cloud-engcomp261.s3-website-us-east-1.amazonaws.com/>

Stack

- React 19 + Vite
- JavaScript puro (sem TypeScript)
- CSS customizado (sem framework de UI)

Telas

Usuários

- Cadastro de novo usuário (POST /users)
- Consulta de usuário por ID (GET /users/{id})
- Listagem de todos os usuários com filtro por nome ou e-mail (GET /users)

Criar Pagamento

- Formulário para criação de pagamento (POST /payments)
- Exibe o resultado com status inicial PENDING

Consultar Pagamentos

- Consulta de pagamento por ID (GET /payments/{id}) com badge de status
- Listagem de pagamentos por User ID (GET /payments?userId=...) com tabela completa

Estrutura

```
frontend/
├── src/
│   ├── App.jsx           ← shell com navegação por abas
│   ├── App.css           ← estilos globais
│   ├── api.js            ← funções de chamada à API
│   ├── assets/
│   │   └── card.svg      ← logo do header
│   └── components/
│       ├── Users.jsx     ← tela de usuários
│       ├── CreatePayment.jsx ← tela de criação de pagamento
│       └── Payments.jsx  ← tela de consulta de pagamentos
├── .env                  ← URL da API (não versionado em produção)
└── vite.config.js
```

Decisões arquiteturais

1. SQS Standard em vez de FIFO

- Ordem não é requisito — pagamentos são independentes entre si.
- Idempotência resolvida via **ConditionExpression** no DynamoDB.
- Throughput ilimitado — FIFO limita a 3.000 msg/s.
- Custo menor por mensagem.

2. DynamoDB em vez de RDS

- Sem servidor para gerenciar — alinha-se com a arquitetura 100% serverless.
- Padrões de acesso simples (busca por chave primária + um GSI), sem JOINS.
- Sem problema de connection pool com Lambda (DynamoDB usa HTTP stateless).
- Escalabilidade automática no modo on-demand.
- Custo menor no MVP — nível gratuito da AWS cobre 25 GB e 25 WCU/mês.

3. AWS Lambda em vez de EC2 ou ECS

- Paga por uso real (milissegundos de execução), não por capacidade ociosa.
- Sem gerenciamento de SO, cluster ou load balancer.
- Escala automática e imediata — cada requisição pode rodar em uma instância separada.
- Integração nativa com API Gateway e SQS via event source mapping.

Trade-off: cold start de 300–800ms quando a função fica inativa. Para mitigar em produção: Provisioned Concurrency.

4. API Gateway REST em vez de HTTP API

- Controle de CORS mais granular por recurso e método — necessário pois o frontend está em domínio S3 diferente da API.
- Conceito de estágios com deploy explícito facilita rastreamento de versões.

- Diferença de custo é desprezível no volume do projeto.

5. EventBridge em vez de invocar consumidores diretamente

- Desacoplamento total — o worker publica e esquece.
- Novos consumidores são adicionados como regras no EventBridge sem alterar o worker.
- Auditoria de domínio via CloudWatch Logs com todos os campos do evento.

6. CloudWatch em vez de ferramenta externa de observabilidade

- Zero configuração de infraestrutura — Lambda, API Gateway, SQS e DynamoDB já enviam métricas automaticamente.
- Integração nativa com alarmes e dashboards configurados em minutos.
- Nível gratuito da AWS cobre 10 métricas customizadas, 5 GB de logs e 3 dashboards/mês.

7. S3 Static Website em vez de servidor web para o frontend

- Sem servidor para gerenciar — S3 serve os arquivos diretamente via HTTP.
- Custo mínimo — armazenamento e transferência de estáticos praticamente zero no volume do projeto.
- Deploy simples: `npm run build` + upload para o bucket.

Consequência: sem HTTPS nativo no Static Website Hosting do S3. Para HTTPS em produção, a solução padrão é CloudFront na frente do bucket.

8. IAM com permissões mínimas por Lambda (least privilege)

- Superfície de ataque reduzida — se uma Lambda for comprometida, o acesso é restrito aos recursos que ela realmente precisa.
- Rastreabilidade via CloudTrail — exatamente qual função executou qual ação em qual recurso.
- Boa prática recomendada pelo AWS Well-Architected Framework.

Testes de carga

Data de execução: 17/05/2026

Ferramenta: Apache JMeter 5.6.3

Região AWS: us-east-1

Arquitetura testada: API Gateway → Lambda (Python) → DynamoDB + SQS → Lambda Worker

Plano A — Rajada de criação (`POST /payments`)

Configuração: 50 threads, 20 loops por thread, ramp-up de 10s → **1.000 requisições totais**

Métrica	Valor
Total de requisições	1.000
Throughput	50,4 req/s

Métrica	Valor
Latência média	505 ms
Latência mínima	442 ms
Latência máxima	1.604 ms
Taxa de erro	0,00%
Duração total	~20s

Observações: A rajada foi absorvida sem erros. O pico de 1.604ms ocorreu no início, provavelmente durante o cold start da Lambda. A latência média de 505ms reflete o custo de três chamadas síncronas em sequência: validação do usuário no DynamoDB + gravação do pagamento + envio para SQS.

Plano B — Consultas simultâneas (`GET /payments/{id}`)

Configuração: 50 threads, 20 loops por thread, ramp-up de 5s → **1.000 requisições totais**

Métrica	Valor
Total de requisições	1.000
Throughput	73,7 req/s
Latência média	431 ms
Latência mínima	398 ms
Latência máxima	556 ms
Taxa de erro	0,00%
Duração total	~14s

Observações: Leituras foram consistentemente mais rápidas e estáveis que escritas — spread de apenas 158ms entre mínimo e máximo, contra 1.162ms no Plano A. O `GET /payments/{id}` faz uma única leitura por chave primária (`GetItem`), sem passar pela SQS.

Plano C — Fluxo misto sustentado (criação + consulta)

Configuração: 30 threads criando pagamentos + 20 threads consultando, por 3 minutos, ramp-up de 15s

Métrica	Valor
Total de requisições	7.621
Throughput	42,2 req/s
Latência média	470 ms
Latência mínima	395 ms

Métrica	Valor
Latência máxima	922 ms
Taxa de erro	0,00%
Duração total	~3min

Observações: Latência média de 470ms ficou entre os valores dos planos A e B, coerente com o mix de operações. Sem degradação progressiva ao longo dos 3 minutos — a arquitetura serverless escala adequadamente sem acúmulo de filas ou esgotamento de conexões.

Análise consolidada

A plataforma sustentou os três cenários **sem nenhum erro** (taxa de erro = 0% em 9.621 requisições totais).

Throughput:

Cenário	req/s
Escrita pura (rajada)	50,4
Leitura pura	73,7
Misto sustentado	42,2

A leitura atingiu throughput ~46% maior que a escrita, o que é esperado dado que envolve menos operações por requisição.

Gargalos identificados:

1. **Cold start da Lambda** — pico de 1.604ms no início do Plano A. Cold starts em Python adicionam ~300–800ms na primeira invocação. Sob carga sustentada (Plano C), esse efeito se dilui.
2. **Operações encadeadas na escrita** — **POST /payments** encadeia três chamadas síncronas (GetItem + PutItem + SendMessage), o que explica a latência ~74ms maior na escrita em relação à leitura.
3. **Scan na listagem de usuários** — **GET /users** usa **scan** sem paginação. Sob volume maior de usuários, esse endpoint degradaria progressivamente.

Pontos positivos:

- Zero erros em todos os cenários — idempotência e validações funcionaram corretamente sob concorrência
- Estabilidade sustentada — sem degradação de latência ao longo dos 3 minutos do Plano C
- Escalabilidade automática — Lambda e DynamoDB (on-demand) absorveram os picos sem intervenção manual
- Desacoplamento via SQS — a API respondeu 202 imediatamente sem aguardar o processamento do worker

Possíveis melhorias futuras

1. Provisioned Concurrency na Lambda da API

Habilitar pelo menos 10 instâncias pré-aquecidas. Eliminaría o cold start para invocações dentro da capacidade provisionada, reduzindo o pico de ~1.600ms para ~500ms no início das rajadas, sem alteração de código.

2. Remover validação síncrona de usuário no **POST /payments**

Delegar essa responsabilidade ao worker. Se o **userId** não existir, o worker rejeita o pagamento com **status=REJECTED**. A API responderia 202 imediatamente, reduzindo a latência média de ~505ms para ~400ms.

3. Confirmar modo on-demand no DynamoDB

Garantir que ambas as tabelas (**Users** e **Payments**) estão em modo on-demand. Em volumes acima de 200 req/s, o modo provisionado poderia ser o principal gargalo por throttling.

4. Aumentar batch size do trigger SQS no worker

Aumentar o batch size para 10 mensagens e habilitar **Maximum Batching Window** de 5s. O worker já itera sobre **event["Records"]**, então nenhuma mudança de código seria necessária. Isso reduziria em 10x o número de invocações do worker e drenaria a fila mais rapidamente sob carga.